

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-1:

- algorytmy ewolucyjne - omówienie zadania realizowanego na laboratoriach,
- pierwsza klasa ag

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025

HOW TO: DRAW A HORSE

BY VAN OKTOP



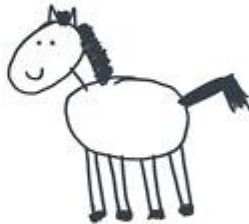
① DRAW 2 CIRCLES



② DRAW THE LEGS



③ DRAW THE FACE



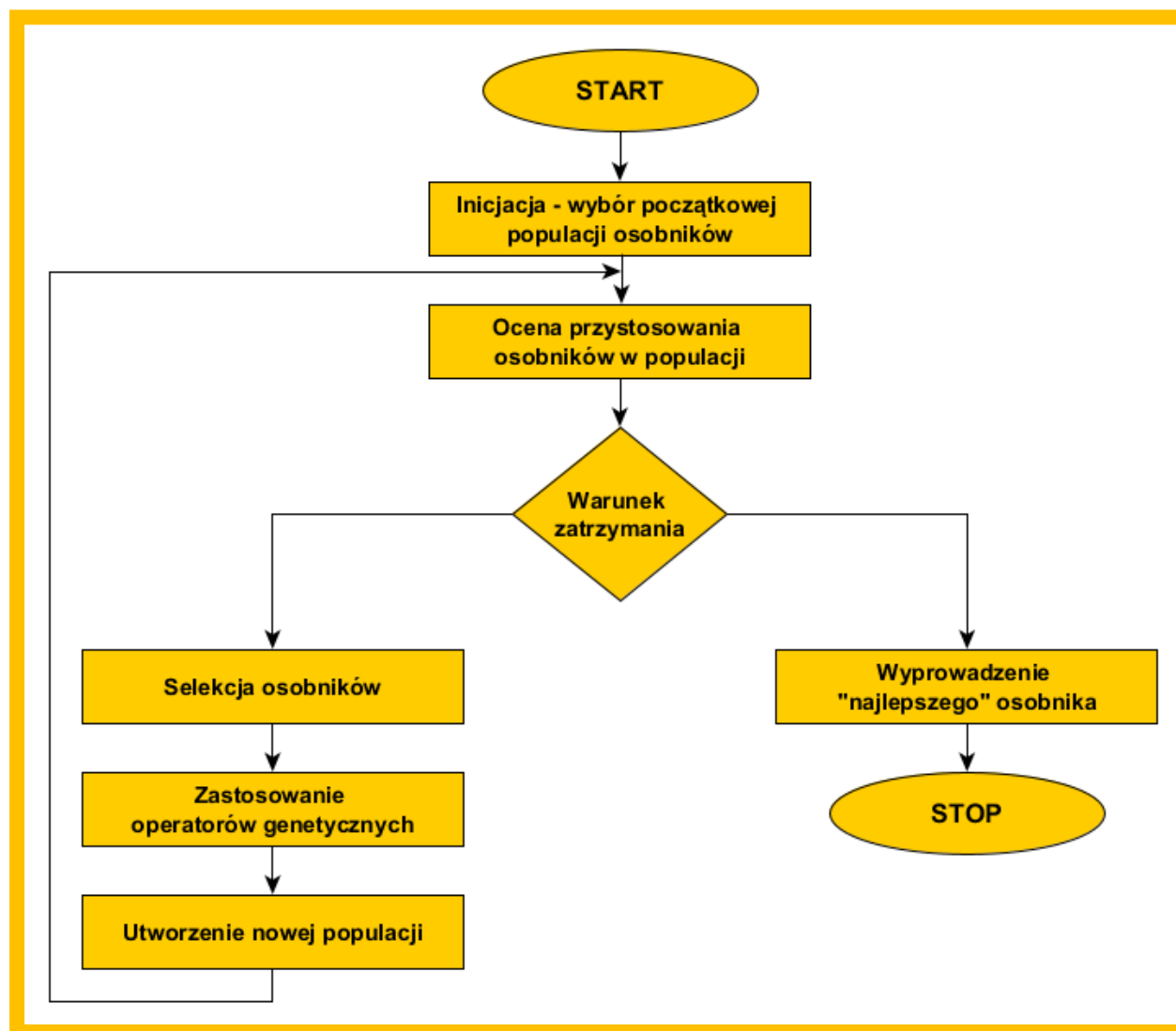
④ DRAW THE HAIR

Projekt kubeczki

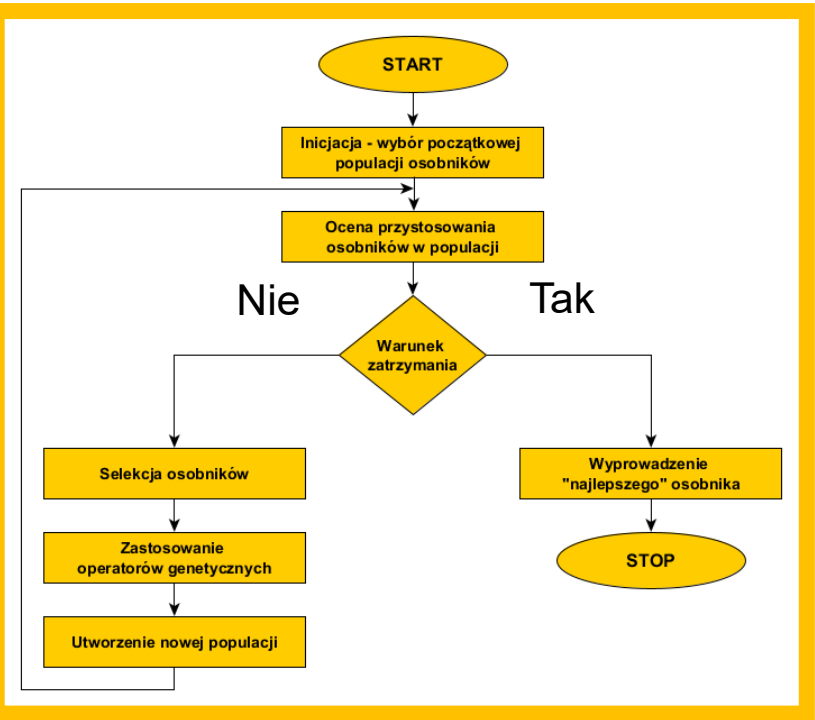


⑤
ADD
SMALL
DETAILS.

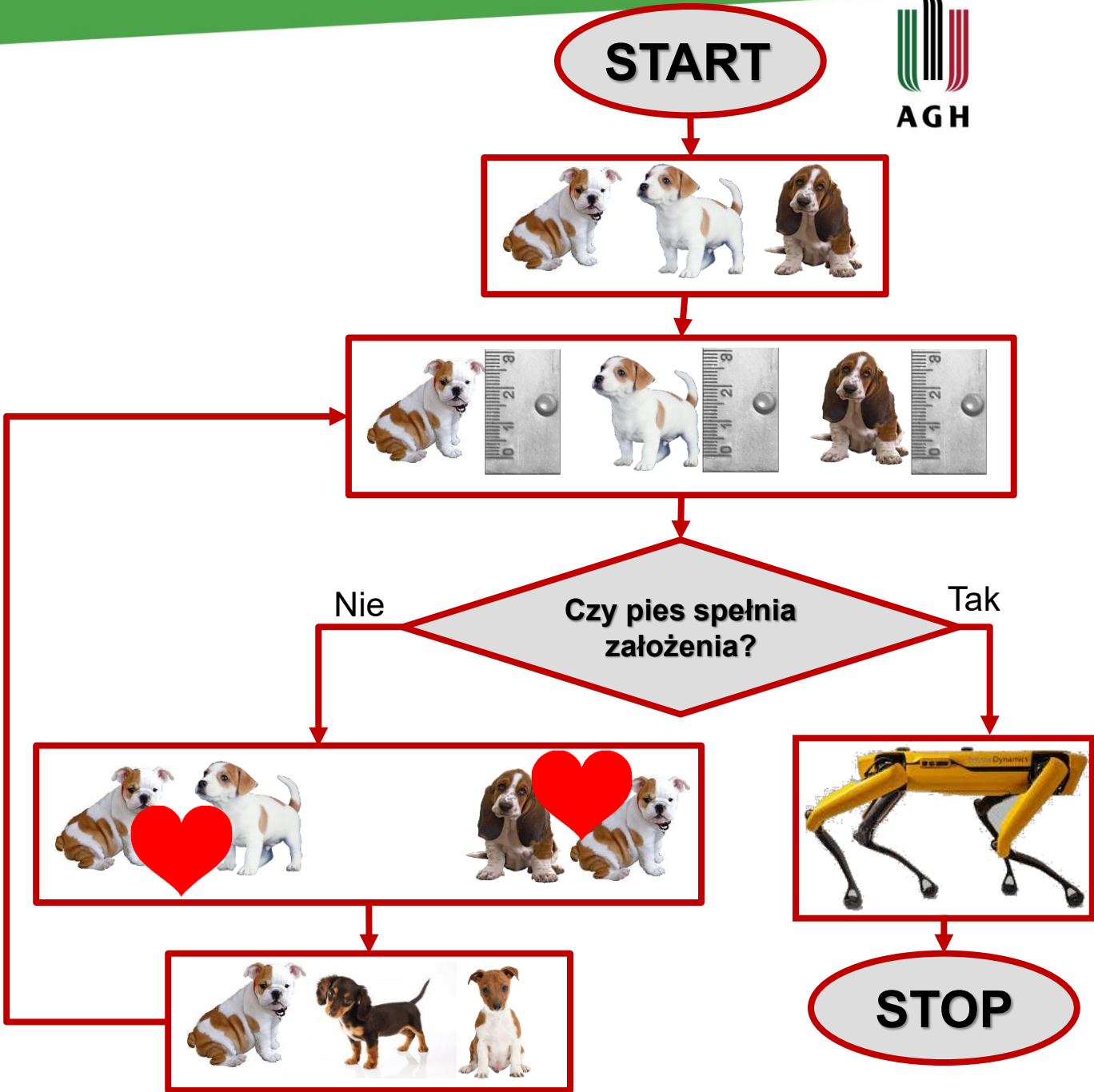
Algorytmy ewolucyjne



Algorytmy ewolucyjne



Funkcja oceny
 wyznacza skalę spełnienia przez osobnika założonych wymagań

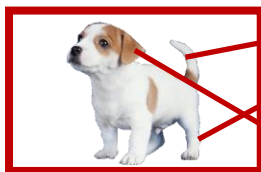




Populacja



Osobnik / chromosom



dł. ogona
dł. łapy
liczba uszu

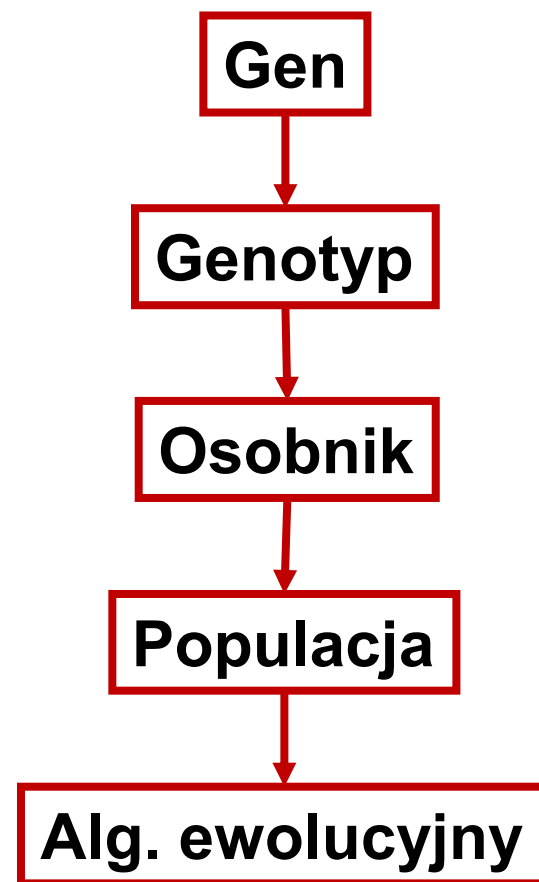
Gen

dł_ogona||dł_łapy||l_uszu

Genotyp

20cm||30cm||2sztuki

Fenotyp

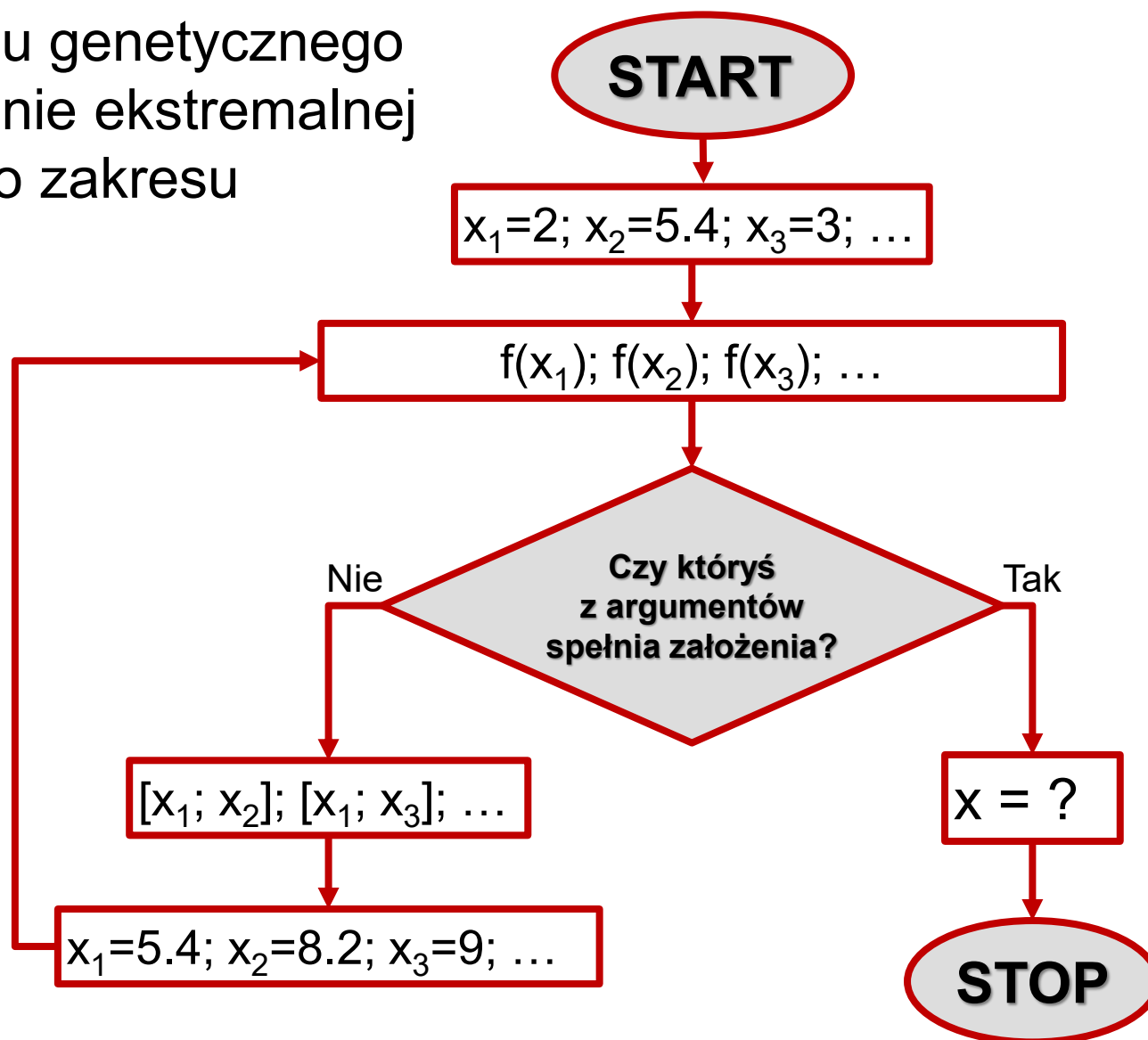


Zadanie realizowane na zajęciach

Cel: implementacja algorytmu genetycznego pozwalającego na wyznaczenie ekstremalnej wartości funkcji dla zadanego zakresu parametrów.

Nudne zadanie?

Raz zaimplementowany algorytm może być zastosowany w dowolnym zadaniu – wystarczy podmienić osobnika oraz funkcję oceny.



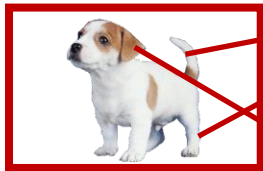
Pierwsza klasa AG



Populacja



Osobnik / chromosom



dł. ogona
dł. łapy
liczba uszu

Geny

dł_ogona||dł_łapy||l_uszu

Genotyp

20cm||30cm||2sztuki

Fenotyp

Gen

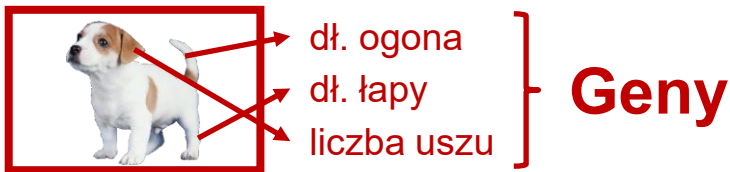
Genotyp

Osobnik

Populacja

Alg. ewolucyjny

Czego oczekujemy od klasy genu?



Gen: dł. ogona

- **id:** [0; 1; 2; 3; ...; N]
 - zakres: [0; 0,5; 1; 1,5; ...; 30] cm
- 
- min krok max**

Gen jest parametrem opisującym osobnika.
W związku z tym musimy mieć narzędzia, które pozwolą pracować z wartością parametru dla analizowanego osobnika.

Potrzebujemy parametrów genu (*klasy*):

- zakres możliwych do osiągnięcia wartości: [min, max, krok];
- wartość parametru dla osobnika – zapisane w postaci pozycji wartości w zakresie: *id*.

Potrzebujemy funkcji, które:

- pozwolą określić wartości parametrów genu – zakres, wartość;
- Pozwolą odczytać wartość parametru osobnika na podstawie pozycji (*id*).

```
1  #pragma once
2
3  #include <string>
4
5
6  class TParam
7  {
8  private:
9      std::string name;
10     double x_start, x_end, dx;
11     int val_id;
12
13 public:
14     TParam(double x_start, double x_end, double dx);
15     TParam(double x_start, double x_end, double dx, double val);
16
17     void set_name(std::string name) { this->name = name; }
18     std::string get_name() { return name; }
19
20     void set_range(double x_start, double x_end, double dx);
21
22     void set_val(double val) { val_id = get_val_id(val); }
23     double get_val() { return x_start + val_id * dx; }
24
25     void info();
26
27 private:
28     int get_val_id(double val);
29
30 };
```

Parametry składowe klasy

Konstruktory klasy

Definicja funkcji składowych

Bonusowa funkcja do wyświetlania stanu obiektu

Na końcu definicji/deklaracji funkcji znajduje się średnik!

```

1  #include <iostream>
2  #include <math.h> // fabs
3
4  #include "TParam.h"
5
6  using namespace std;
7
8  TParam::TParam(double x_start, double x_end, double dx)
9  {
10     set_range(x_start, x_end, dx);
11     name = "";
12     val_id = 0;
13 }
14
15 TParam::TParam(double x_start, double x_end, double dx, double val)
16 {
17     set_range(x_start, x_end, dx);
18     set_val(val);
19     name = "";
20 }
21
22 void TParam::set_range(double x_start, double x_end, double dx)
23 {
24     this->x_start = x_start;
25     this->x_end = x_end;
26     this->dx = dx;
27 }
28
29 int TParam::get_val_id(double val)
30 {
31     if (val < x_start) return 0;
32
33     else if (val > x_end) return (x_end - x_start) / dx;
34
35     else
36     {
37         double x = x_start;
38         int _id = 0;
39
40         while (fabs(x + _id * dx - val) > dx / 2) _id++;
41
42         return _id;
43     }
44 }

```

```

46 void TParam::info()
47 {
48     cout << "\n";
49     cout << "=====\n";
50     cout << "== name: " << name << "\n";
51     cout << "== range: [" << x_start << "; " << x_end << "; " << dx << "]\n";
52     cout << "== value: " << get_val() << " \t (id: #" << val_id << ")\n";
53     cout << "=====\n\n";
54 }

```

TParam.cpp

```
1 #include <iostream>
2
3 #include "TParam.h"
4
5 using namespace std;
6
7
8 int main()
9 {
10     TParam param1{ 1, 4, 1, 2 };
11     TParam param2{ 10, 20, 3 };
12     TParam param3{ 0, 10, 0.5, 3.3 };
13
14     // //////////////////////////////////////
15
16     cout << "param1";
17     param1.info();
18
19     cout << "param2";
20     param2.info();
21
22     cout << "param3";
23     param3.info();
24
25     // //////////////////////////////////////
26
27     param2.set_val(100);
28     param3.set_val(7.5);
29
30     cout << "=====\n";
31     cout << "AFTER\n";
32     cout << "=====\n\n";
33
34     cout << "param2";
35     param2.info();
36
37     cout << "param3";
38     param3.info();
39
40     // //////////////////////////////////////
41
42     return 0;
43 }
```



```
param1
=====
== name:
== range: [1; 4; 1]
== value: 2      (id: #1)
=====

param2
=====
== name:
== range: [10; 20; 3]
== value: 10     (id: #0)
=====

param3
=====
== name:
== range: [0; 10; 0.5]
== value: 3.5    (id: #7)
=====

=====
AFTER
=====

param2
=====
== name:
== range: [10; 20; 3]
== value: 19     (id: #3)
=====

param3
=====
== name:
== range: [0; 10; 0.5]
== value: 7.5    (id: #15)
=====
```

Zadania do samodzielnego wykonania*

1. Modyfikacja funkcji składowych:

- dodanie dwóch nowych konstruktorów pobierających wartość zmiennej *nazwa*.

```
6  class TParam
7  {
8  private:
9      std::string name;
10     double x_start, x_end, dx;
11     int val_id;
12
13 public:
14     TParam(double x_start, double x_end, double dx);
15     TParam(double x_start, double x_end, double dx, double val);
16     TParam(std::string name, double x_start, double x_end, double dx);
17     TParam(std::string name, double x_start, double x_end, double dx, double val);
18
19     void set_name(std::string name) { this->name = name; }
20     std::string get_name() { return name; }
21
22     void set_range(double x_start, double x_end, double dx);
23
24     void set_val(double val) { val_id = get_val_id(val); }
25     double get_val() { return x_start + val_id * dx; }
26
27     void info();
28
29 private:
30     int get_val_id(double val);
31
32 };
33
```

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-1:

- algorytmy ewolucyjne - omówienie zadania realizowanego na laboratoriach,
- pierwsza klasa ag

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025